

提问体验核查与改进方案

芝士的「带按钮提问」与 Codex 提问体验的差距 | 2026-09-30

一句话结论：现在的问题是「一条聊天消息下面挂两三个按钮，点一下就发出去」。它够用，但比你在 Codex 里用惯的提问少了三样东西——**没有「写别的」的入口、没有「选完再提交」这一步、没有「先不答」**。这三样加起来，就是「显示和操作不直观」的主要来源。下面是逐条证据和一份只加不改的最小方案。

本文件里所有界面图为示意图，用编造的演示内容画成，不是线上真实记录；界面画法（按钮、颜色、圆角、字级）取自产品现有样式。

一、现在一次提问是怎么走的

从头到尾五步

1. 你让芝士做事。它中途需要你拍板，就发出一条提问：一句话，加上两到四个可点的选项。
2. 这条提问出现在聊天里，**和普通消息长得一样**，选项是它下面的一排按钮。
3. 你在按钮上点一下，答案**立刻**发出去，没有确认。
4. 芝士收到答案，下一轮接着往下做。聊天里留下一行「某某 选了『…』」。
5. 如果你不点，提问就一直挂着。没有过期、没有催办、也没有人知道它晾了多久。

两处需要你知的限制

想说选项以外的话，只能放弃按钮。三个选项都不合适时，界面上没有地方写别的。唯一的办法是不管这些按钮，改到下面的聊天框里打字——而你打的那段话在界面上与这道题没有任何关联，看起来像你自己另起了一个话题。

点错不能改。一道题只能答一次。答完想换，界面上没有「改」这个动作。

二、三个时刻的界面对照

同一个问题的三个时刻：刚问出来、正在回答、已经回答。上：现在的产品样子。下：建议的最小改动。方框里是示例数据，不是线上真实记录。

现在的产品样子 点一下按钮就发出去，中间没有停顿；没有「写别的」和「先不答」

① 刚问出来

芝芝士 14:02

数据用哪一份？

课程发的那份 sales.csv 项目里的演示数据

问题就是聊天里的一条消息，按钮挂在下面。**没有标题，也没有任何标记说明「这题在等你答」**，它看起来和一句普通说话一样。

② 点下去的那一瞬间

芝芝士 14:02

数据用哪一份？

课程发的那份 sales.csv 项目里的演示数据

鼠标点下的那一刻答案就已经发出去了。**没有「选中了，确认一下」这一步**，手一滑点错也没法收回。这一格与其说是「回答中」，不如说是「已经交了、正在送达」。

③ 已经回答

芝芝士 14:02

数据用哪一份？

✓ 王长鑫 选了「课程发的那份 sales.csv」

回答之后会留下一行「谁选了什么」，所有人都看得见，这一点是好的。但**不能改答案**；而且另一处（首页「待我处理」）答完是整条直接消失，什么都不留。

建议的最小改动 多两样东西：一个「写别的」的输入框，和一个「提交」按钮

① 刚问出来

芝芝士 14:02

数据用哪一份？

课程发的那份 sales.csv 项目里的演示数据

或者写一句别的...

提交回答 先不答 等你回答

选项照旧，下面多一个「**或者写一句别的**」的输入框，两个选项都不合适时不用改道去聊天框打字。右边一行小字「**等你回答**」，一眼看得出这不是一条普通消息。

② 选好了，还没交

芝芝士 14:02

数据用哪一份？

课程发的那份 sales.csv 项目里的演示数据

或者写一句别的...

提交回答 先不答 已选，按「提交回答」发出

点选项只是**选中**，可以反悔、可以换，也还可以改成写一句别的。按下「提交回答」才发出去。手滑点错不再是不可逆的事。

③ 已经回答

芝芝士 14:02

数据用哪一份？

✓ 王长鑫 选了「课程发的那份 sales.csv」

已交给芝士，它下一轮会接着往下做 · 改答案

留下「谁选了什么」不变，但下面多一句「**已交给芝士，它下一轮会接着往下做**」，让你知道对方收到了。另给一个「**改答案**」，答错能自己救回来，不用打字解释。「待我处理」里答完也改成留这一行，而不是整条消失。

三、逐条差距

「现在」一栏读自产品代码与真实记录；「更顺手的形态」一栏是本方案要达到的样子。需要说明的是：OpenAI 与 Claude Code 的官方文档页（2026-09-30 查过）都没有写 提问工具的交互细节，所以这一栏不是照抄某份官方说明。凡是没有可核对来源的行为，本文件没有写。

问题怎么显示

现在：就是聊天里一条普通消息，选项挂在下面。没有标题，也没有任何标记说明「这题在等你答」。

更顺手的形态：一眼看得出这是一道在等你回答的题，而不是谁随口说的一句话。

一次问几件事

现在：一次只能问一件事。要问三件事就得发三条，中间还隔着别的消息。

更顺手的形态：一次可以把几个相关的短问题摆在一起答完。

选项以外怎么说

现在：没有入口。想说别的只能放弃按钮，改去聊天框打字，那句话和这道题在界面上不关联。

更顺手的形态：选项旁边留一个「写别的」的地方，不用换地方。

选完就交，还是选完再交

现在：点一下就发出去，没有停顿，手滑点错不可逆。

更顺手的形态：选中和提交是两步，选完可以反悔、可以换。

暂时不想答

现在：没有这个动作。问题就一直挂着。

更顺手的形态：可以放着，而且界面上标明它在等谁。

答完之后

现在：聊天里留一行「谁选了什么」；但另一处入口（首页「待我处理」）答完整条消失，什么都不留。

更顺手的形态：两处都留下清楚的回显，并且能看到对方确实收到了。

答错之后

现在：不能改。

更顺手的形态：可以改，或者补一句说明。

四、两类问题分开

按仓库的约定：产品没做到它自己声称要做到的事（包括说了不实的话）算缺陷，可以直接修；**改变你看到什么、能做什么要先给你看过、你点头才能合**。下面按这条分开。

A. 缺陷，可以直接修

1. **说明文字与实际行为不符**。提问接口的说明写着「本轮停在这里等回答」，但实际行为是「问完就收尾，不等回答」，集成测试也是这么断言的。说明是过时的，会误导后来的人。→ 改说明，不动功能。
2. **两处提问入口答完之后行为不一致**。聊天里的提问答完会**立刻叫醒**提问方；首页「待我处理」里的提问答完**不叫醒**，只往聊天里丢一条没有任何人被点名的记录。要芝士下次因为别的事被叫起才读得到这条回答——很可能永远读不到。→ 让后者也叫醒提问方。这属于「产品没做到它声称要做到的事」。
3. **「待我处理」里答完什么都不留**。聊天里答完留「谁选了什么」，这一处答完整条消失。→ 留同一行回显。

B. 改版，要你点头才能做

加三样东西，只加不改，现有操作一个都不动：

1. **「或者写一句别的…」输入框** —— 选项不合适时能直接说自己的话。
2. **「提交回答」按钮** —— 选中和提交分成两步，点错能反悔。
3. **「先不答」按钮** —— 暂时答不了可以放着，问题标成「等你回答」。

另有两件可以第二批再议：答完给一个「改答案」；一次可以问多个短问题。

五、为什么会有这个差距

这个差距不是「少做了几个按钮」，它有一个真实的起因：

你在 Codex 里用的那个提问工具，是 Codex 自己带的、直接对着你弹的。芝士的情况不一样——芝士是**在一个房间里替你干活**，它跑在一个后台进程里，它自己带的提问工具在房间里**没有人能回答**：那个提问会变成一个挂在后台进程上的空等，房间里谁也看不见。所以平台把它关掉了，换成现在这个「聊天里发按钮」。

这个换法本身是对的，问题在于换过来的这个太简陋：只留下「选一个按钮」这一条路。本文件第二、三节要补的，就是把被换掉的那些能力补回来。

这一段的依据：仓库里有一张对每个助手逐一核对过的能力表（对着锁定的程序版本读出来的，版本一变就要重核）。表上记着：Codex 的提问工具有两个，其中一个关不掉（记为已知缺口）；Claude Code 的提问工具两侧都拒。表格本身在工程附注里。

六、下一步

1. 请你先看第二节的图，确认这个方向是不是你要的。图上标着「①②③」三个时刻，下排是建议形态。不对就直接说，改图比改代码便宜。
2. **A 类缺陷我直接修**，走正常的代码评审和上线流程，不再来问你。
3. **B 类改版等你点头再动**。点头之后我也只是按图上这一版做，不自行加东西。

工程附注（不看也不影响上面的结论）

关键位置，便于复核：提问入口 `backend/app/api/routes/topics.py` 的 `ask_options`（约 1828 行）与 `answer_options`（约 2045 行）；按钮渲染 `frontend/src/components/room/RoomMessage.vue`（约 266 行）；另一处入口 `backend/app/domain/notification/services.py` 的 `ProjectNotificationService.resolve`（约 383 行）；能力表 `backend/app/domain/agent/harness/codex/behaviour.py` 与 `claude_code/behaviour.py`，汇总在 `docs/manual/dev/harness.md`。

外部资料：查过 OpenAI Codex 文档（developers.openai.com/codex/cli，2026-09-30）与 Claude Code 文档（code.claude.com/docs/en/interactive-mode，2026-09-30），**两处都没有写提问工具的交互细节**。所以本文件第三节「更顺手的形态」一栏写的是交互上应当具备的形态，不是照抄某份官方说明；凡是没有可核对来源的行为，本文件没有写。

复现方式：全部结论读自仓库代码、集成测试与一张真实话题文档的记录，**没有在任何活跃话题里发过测试问题**。第二节的图是独立画的示意图。